

Optimizing Transaction Schedules on Universal Quantum Computers via Code Generation for Grover's Search Algorithm

Sven Groppe

Institute of Information Systems (IFIS)
University of Lübeck
Lübeck, Germany
groppe@ifis.uni-luebeck.de

Jinghua Groppe

Institute of Information Systems (IFIS)
University of Lübeck
Lübeck, Germany
groppej@ifis.uni-luebeck.de

ABSTRACT

Quantum computers are known to be efficient for solving combinatorial problems like finding optimal schedules for processing transactions in parallel without blocking. We show how Grover's search algorithm for quantum computers can be applied for finding an optimal transaction schedule via generating code from the problem instance. We compare our approach with existing approaches for traditional computers and quantum annealers in terms of preprocessing, runtime, space and code length complexity. Furthermore, we show by experiments the expected number of optimal solutions of this problem as well as suboptimal ones. With the help of an estimator of the number of solutions, we further speed up our optimizer for optimal and suboptimal transaction schedules.

CCS CONCEPTS

• Information systems → Data management systems; Database transaction processing.

KEYWORDS

Quantum computing, transaction processing, synchronization, 2-phase-locking, database, schedule

ACM Reference Format:

Sven Groppe and Jinghua Groppe. 2021. Optimizing Transaction Schedules on Universal Quantum Computers via Code Generation for Grover's Search Algorithm. In *25th International Database Engineering & Applications Symposium (IDEAS 2021)*, July 14–16, 2021, Montreal, QC, Canada. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3472163.3472164>

1 INTRODUCTION

For the universal quantum computer, Grover [14] developed an algorithm for finding (with a high probability) the input to a black box function with a particular result to be searched for. In comparison to traditional computing, Grover's search algorithm achieves a quadratic speedup, which is to be optimal among all possible quantum algorithms [3, 14].

Grover's search often serves as basis for algorithms solving combinatorial optimization problems [2] by

- encoding candidates of the solution as an integer number, which is the input of the black box function of Grover's search, and
- using a function to check if a candidate solution has costs below a given threshold as black box function for Grover's search.

We analyze this pattern of solving combinatorial optimization problems on quantum computers with the optimizing transaction schedules problem [5, 6]. The optimizing transaction schedules problem is a variant of the job shop scheduling problems (JSSP), where jobs are assigned to machines (i.e., cores of a multi-core CPU in the domain of parallel computing jobs) with the optimization goal to minimize the overall processing time. For the optimizing transaction schedules problem, the jobs are transactions, which might block each other because of accessing at least one same data object, and conflicting transactions are scheduled to **not** run in parallel avoiding blocking of these transactions.

We propose to optimize the transaction schedule of batches by utilizing quantum computers as hardware accelerators [11–13]. By using pipelining we can optimize the transaction schedule already of the next batch during the processing of the current batch of transactions (see Figure 1). In this way, the quantum computer is run in parallel to the transaction processing on traditional CPUs in order to minimize waiting times and to maximize throughput.

Grover's search needs to call its black box function with candidate solutions in superposition. Only a limited set of operations can be applied to qubits in superposition, such that not all algorithms of traditional computing have corresponding counterparts in quantum computing. As a consequence for many combinatorial optimization problems and also for the optimizing transaction schedules problem, the code (i.e., the combination and orchestration of the quantum logic gates) of the black box function needs to be generated for the specific instance of the considered problem. We will show how to generate the code of the black box function for an instance of the optimizing transaction schedules problem.

Whenever the black box function doesn't need constant time, but its runtime depends on the instance of the considered problem, the overall runtime complexity of solving the considered combinatorial optimization problem is higher than the runtime complexity of the pure Grover's search. Hence, we will analyze the runtime complexity of solving the optimizing transaction schedules problem. Additionally, we analyze the preprocessing time (needed for generating the black box function), space requirements (i.e., number of qubits) and code length, and compare these results with the naive implementation on traditional computers (enumerating and testing all candidate solutions) and a recent solution [5, 6] on quantum annealers (specialized quantum computers for solving quadratic unconstrained binary optimization (QUBO) problems).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IDEAS 2021, July 14–16, 2021, Montreal, QC, Canada

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-8991-4/21/07...\$15.00

<https://doi.org/10.1145/3472163.3472164>

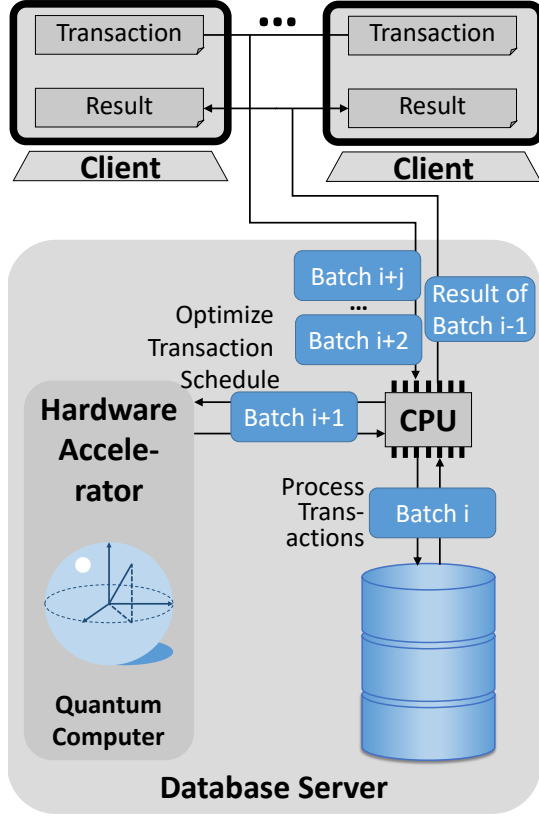


Figure 1: Pipelining the optimization of transaction schedules and the processing of transaction batches: The batches $i + 2$ to $i + j$ of transactions are in the queue for optimizing their schedule and processing, the schedule of batch $i + 1$ is currently optimized by the quantum computer, batch i is currently processed and the results of batch $i - 1$ are being transmitted to the clients.

We have implemented an optimizer for transaction schedules by generating the code for the quantum simulator Silq [4] to show that the code generation is quite efficient and tests of the generated code are successful in Silq.

Our main contributions are

- a code generation approach for the black box function of Grover’s search running on universal quantum computers for optimizing transaction schedules,
- a complexity analysis concerning preprocessing and execution time, space and code length, and comparison of our approach with the ones running on traditional computers and on quantum annealers, and
- an implementation for the quantum simulator Silq [4].

2 BASICS

In this section, we introduce quantum computing in subsection 2.1, Grover’s search algorithm in subsection 2.2 and transaction management in subsection 2.3.

2.1 Quantum computer

Quantum computers are computers that are not based on classical mechanics, instead they exploit the effects of quantum mechanics.

Quantum mechanics describes the states and behavior of particles that are smaller than the size of an atom and do not follow the laws of classical physics. At this scale, there occur effects that the quantum computer makes use of, especially the principle of superposition and that of quantum entanglement. The quantum computer uses qubits, which can take on 2 states simultaneously due to the principle of superposition. While a bit can assume the state 0 or 1, a qubit assumes the states 0 and 1 simultaneously. If a measurement of the state is made, the qubit changes to one of the two states. Both states have relative probabilities with which they are assumed in the measurement. The principle of quantum entanglement enables the mutual influence of qubits, since entangled qubits mutually influence their probabilities. Imagining the principle of superposition as a special form of parallel computing opens up a new world of computation beyond polynomial time and allows in theory an exponential speedup compared to classical computers.

Universal Quantum computing finds desired solutions of a problem by clever manipulation of single qubits as well as entangled qubits. For the purpose of manipulating qubits, gates provide elementary operations on one or two qubits. For example, the Hadamard-Gate puts one qubit into superposition and a Controlled-NOT(CNOT)-Gate inverts a second qubit depending on the first [1]. A quantum computer is thus able to execute quantum algorithms like Shor’s algorithm for factorizing large numbers [19] or Grover’s algorithm for searching in huge unsorted databases [14, 15].

2.2 Grover’s search algorithm

Grover [14] showed that his search algorithm has an expected runtime of $\frac{\pi}{4} \cdot \sqrt{N}$ basic steps and claimed based on a result in [3] that up to a multiplicative constant among all possible quantum algorithms the optimality of his algorithm can be proven. Variants [7, 18] of Grover’s search propose applications running on average in $u \cdot \sqrt{\frac{N}{k}}$ basic steps without knowing the number k of solutions in advance. The variants differ upon being randomized [7] and deterministic [18], and the constant u , where [7] achieves $u = \frac{9}{4}$ and [18] $u = \frac{8 \cdot \pi}{3}$. Whenever the number k of solutions is approximately known in advance, then Grover’s search is able to find a solution in $\frac{\pi}{4} \sqrt{\frac{N}{k}}$ basic steps, such that a speedup of $\sqrt{k}$ can be achieved in comparison to when only one solution is present, a speedup of 2.86 in comparison to [7] and 10.66 in comparison to [18].

2.3 Transaction Management

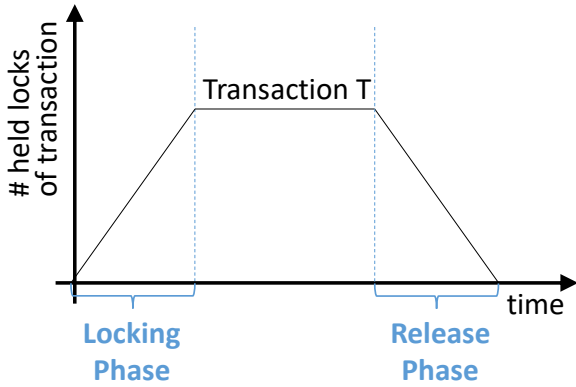
A **transaction** $t = \langle s_1, \dots, s_{|t|} \rangle$ of length $|t|$ is a series of operations s_i , carried out by a single user or application program, which reads or updates the contents of the database [9]. Each operation $s_i = a_i(e_i)$ consists of the type of access $a_i \in \{r, w\}$, where r represents a read access and w a write access, and the object e_i to be accessed.

For example, the transaction $t = \langle r(A), w(A), r(B), w(B) \rangle$ is of length $|t| = 4$.

Since transactions often need to access a database at the same time and thereby often reference the same objects, transactions in a database system are required to fulfill the so-called ACID properties [9]. The focus here is on the fulfillment of the (I)solation property, which guarantees to avoid problems of unsynchronized parallel execution of several transactions and requires concurrently executed transactions to not influence each other.

2.3.1 Conflict Management. To ensure the isolation property, conflicts between transactions must be dealt with. The simplest strategy to deal with conflicts would be to execute the transactions serially. Since this is too slow for operational systems and also many transactions do not conflict with each other at all, in practice transactions are executed in parallel. There are various approaches to dealing with the conflicts that arise, one of them is the use of locks. For this purpose, each transaction acquires a lock for an object before access and releases it after access, thus preventing concurrent access to an object.

a) 2 Phase Locking



b) Strict Conservative 2 Phase Locking

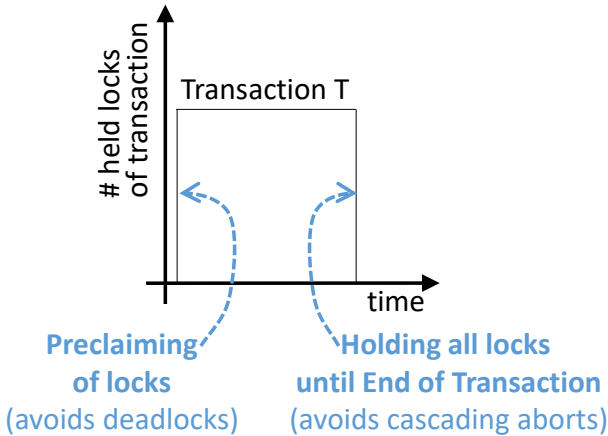


Figure 2: a) Two-phase-locking protocol with locking phase and release phase, and b) the strict conservative two-phase locking protocol

The **two-phase-locking protocol** (see Figure 2) is a locking protocol that requires each transaction consisting of two subsequent phases, the locking phase and the release phase. During the locking phase, the transaction may acquire locks but not release them, whereas the release phase requires the release of previously required locks. If a transaction has released a lock, it may not acquire any new ones. There is a distinction between *read locks* (also called *shared locks*) and *write locks* (also called *exclusive locks*). Variants of the protocol include the conservative two-phase-locking protocol and the strict two-phase-locking protocol. The conservative variant (*preclaiming*) requires that all locks that are required during a transaction are acquired before the transaction is started.¹ The strict variant holds all locks until the end of a transaction, which avoids *cascading aborts* occurring in case of so called *dirty reads* of objects written by transactions, which are later aborted resulting in an abort of the current transaction as well. In this contribution, we consider the combination of the conservative and the strict two-phase-locking protocol in our transaction model, which results in a serial execution of all transactions that access the same objects.

2.3.2 Conflicts. Let T be the set of transactions, D be the set of data objects. Two transactions $i \in T$ and $j \in T$ are in conflict with each other if there exists two operations of these transactions being in conflict with each other. Two operations $a_i(e) \in i$ and $a_j(e) \in j$ of the transactions i and j are in conflict with each other if they access the same object $e \in D$ and at least one of the operations is writing the object:

$$\begin{aligned} & a_i(e) \text{ in conflict with } a'_j(e) \text{ if} \\ & \exists i, j \in T, e \in D, a_i(e) \in i, a'_j(e) \in j : \\ & i \neq j \wedge (a_i = w \vee a'_j = w) \end{aligned}$$

We assume that accesses to the same object $e \in D$ are serialized and we use the notation $a_i(e) \rightarrow a'_j(e)$ in order to denote that the operation $a_i(e)$ is executed before $a'_j(e)$. The isolation property is therefore fulfilled if all conflict operations of conflicting transactions $i \in T$ and $j \in T$ ($i \neq j$) are processed in the same order of transactions:

$$\begin{aligned} & \left(\forall a_i(e) \in i, a'_j(e) \in j : \right. \\ & \quad \left. (a_i = w \vee a'_j = w) \Rightarrow a_i(e) \rightarrow a'_j(e) \right) \\ & \quad \vee \\ & \left(\forall a_i(e) \in i, a'_j(e) \in j : \right. \\ & \quad \left. (a_i = w \vee a'_j = w) \Rightarrow a'_j(e) \rightarrow a_i(e) \right) \end{aligned}$$

The two-phase-locking protocol fulfills the isolation property by guaranteeing that each object is continuously locked from its first to last access and during the entire processing. An operation $a_i(e)$ of a transaction i is suspended if another transaction j ($i \neq j$) already holds a lock to the object e (and the held lock or the lock to be acquired is an exclusive lock). In this way it is guaranteed that all conflicting operations of transaction j are executed before those

¹Please note that for those transactions, for which the required locks are not known before processing, the required locks can be determined by an additional phase before transaction processing. The contribution in [20] describes such an approach which can be also applied in our scenario.

operations of transaction i . When using preclaiming of locks, according to [20] there are no real limits (by determining the required locks before transaction processing in an additional phase), while the occurrence of deadlocks is eliminated, where transactions wait for releasing the locks of each other.

2.3.3 Optimal Transaction Schedules without Blocking. For an example of transactions, see Figure 3, which block each other as in Figure 4. In our example, we want to run these transaction on multi-core CPU with three cores (see Figure 5). In optimal solutions, transactions blocking each other do not run in parallel. We present one optimal solution in Figure 6. We already see that there will be many more optimal solutions. For example, by just exchanging the cores, e.g., core 2 runs the transactions of core 3 in Figure 6 and vice versa, we already get $3! = 6$ equivalent transaction schedules. Hence, we recognize that there will be many optimal solutions, which speeds up Grover’s search algorithm.

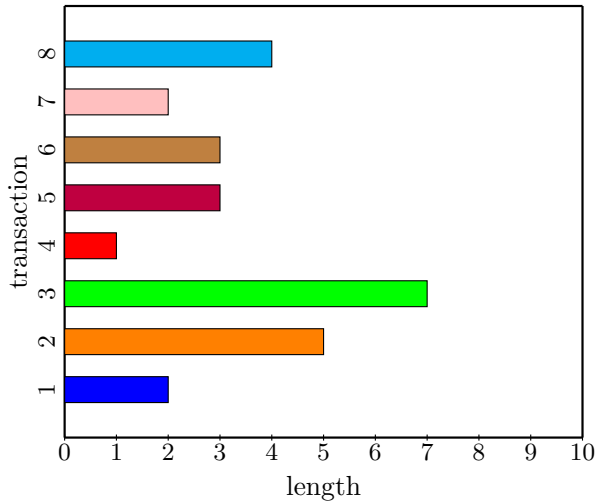


Figure 3: $n = 8$ transactions with their respective lengths

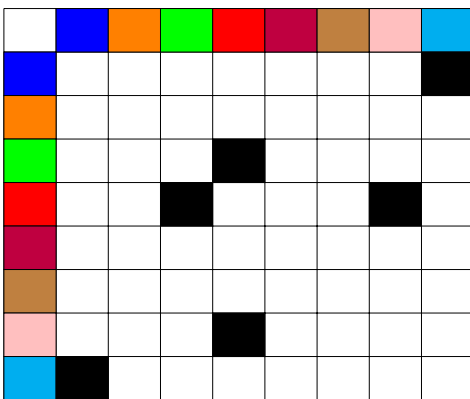


Figure 4: Black fields indicate blocking transactions of the transactions of Figure 3

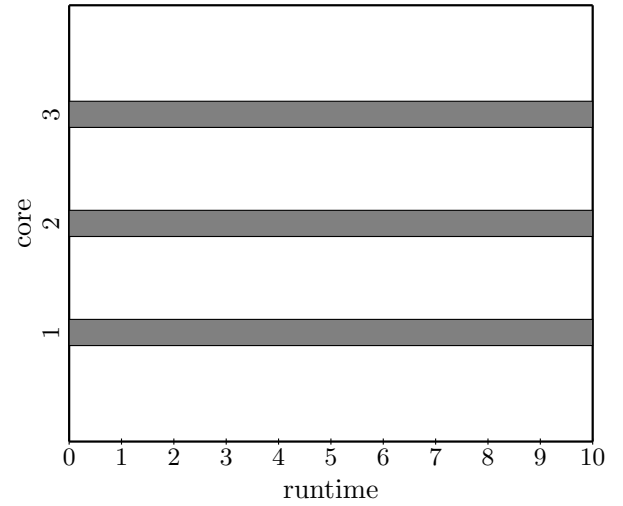


Figure 5: Transactions are scheduled on three cores of a multi-core CPU, the maximum runtime here is 10 time units

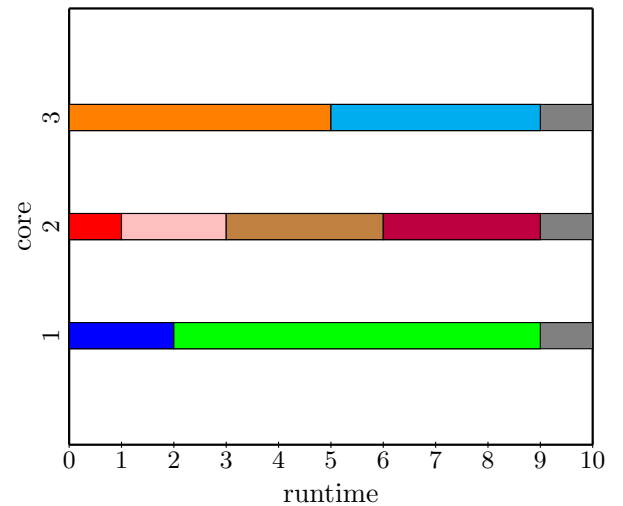


Figure 6: One optimal solution of a transaction schedule for the transactions of Figure 3, which block each other as in Figure 4, on three cores of a multi-core CPU without any blocking with a runtime of 9 time units

3 OPTIMIZING TRANSACTION SCHEDULES BY UNIVERSAL QUANTUM COMPUTERS

We propose the following approach for optimizing transaction schedules on universal quantum computers: The code for the black box function for Grover’s search is generated (see subsection 3.2) based on the transaction lengths and conflicts between the transactions (see Figure 7). Because Grover’s search needs less iterations for those problem instances with more than one solution, we estimate the number of solutions (see subsection 3.3) for our considered transactions in order to speed up our approach. The input of the

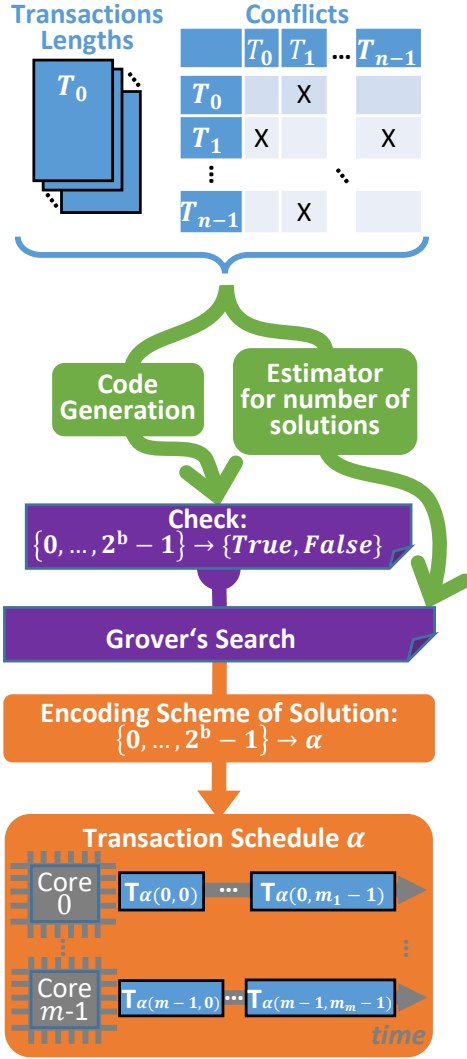


Figure 7: Overview of the code generation approach for optimizing transaction schedules.

black box function for Grover's search is an integer number in superposition of a given number of qubits. In our approach, the input number in superposition represents possible transaction schedules and Grover's search returns one integer number of those which successfully passed the black box function. Hence we need a mapping of a given integer number to transaction schedules α , which is also called *encoding scheme* for transaction schedules (see subsection 3.1).

3.1 Encoding Scheme for Transaction Schedules

We describe the representation of a transaction schedule, i.e., the sequence of transactions for each machine (i.e., core of a multi-core CPU), by an integer number and determine the number of used bits.

Algo determineSchedule
Input: $p: \{0, \dots, 2^b - 1\}$
Output: $\{0, \dots, n-1\}^{m-1} \times \{0, \dots, n-1\}^{n-1}$

```

for(x in 1..m-1)
     $\mu_x = p \bmod n$ 
     $p = p \div n$ 
 $a = [0, \dots, n-1]$ 
for(i in 0..n-1)
     $j = p \bmod (n-i)$ 
     $p = p \div (n-i)$ 
     $\pi[i] = a[j]$ 
     $a[j] = a[n-i-1]$ 
return  $(\mu_1, \dots, \mu_{m-1}, \pi)$ 
    
```

Example (n=4, m=2):
 29

```

x = 1:
 $\mu_1 = 1$ 
p = 7
a = [0, 1, 2, 3]
i = 0: | i = 1: | i = 2: | i = 3:
j = 3 | j = 1 | j = 0 | j = 0
p = 1 | p = 0 | p = 0 | p = 0
 $\pi[0] = 3$  |  $\pi[1] = 1$  |  $\pi[2] = 0$  |  $\pi[3] = 2$ 
a[3]=a[3] | a[1]=a[2] | a[0]=a[1] | a[0]=a[0]
return (1,[3,1,0,2])
    
```

Figure 8: Algorithm for determining a transaction schedule from an integer number with example

We propose to split the representation of a transaction schedule into two components: The first component represents a permutation of the given transactions and the second component determines where to split the permutation of transactions into m subsequences to be scheduled to the m machines.

Because there are $n!$ possibilities for a permutation π of n items, our domain for representing the permutation of n transactions is $\{0, \dots, n! - 1\}$ for which we need $\lceil \log_2(n!) \rceil$ bits. For decoding $p \in \{0, \dots, n! - 1\}$ to a permutation π of transactions, we use a variant [8] of Lehmer's code [16] with runtime complexity $O(n)$.

Representing a permutation of the n transactions into m subsequences to be scheduled to the m machines, we need $m - 1$ markers μ . We propose to use the domain $\{0, \dots, n-1\}$ for each of the markers and assume that at least one transaction must be processed on the first machine. We require $0 \leq \mu_1 < \mu_2 < \dots < \mu_{m-1} \leq n-1$, such that otherwise it is not a valid transaction schedule to be discarded by the generated black box function of Grover's search. Defining $\mu_0 := -1$ and $\mu_n := n-1$, the scheduled transactions on machine m_x are $\langle \pi(\mu_{m_{x-1}} + 1), \dots, \pi(\mu_{m_x}) \rangle$. The function $\alpha(x, i)$ to determine the i -th transaction on machine x is hence defined as $\alpha(x, i) = \pi(\mu_{m_{x-1}} + 1 + i)$, where $x \in \{0, \dots, m-1\}$ and $i \in \{0, \dots, \mu_{m_x} - \mu_{m_{x-1}} - 1\}$.

We present the complete algorithm for determining a transaction schedule from an integer number with example in Figure 8.

For each marker we need $\lceil \log_2(n) \rceil$ bits, such that we overall need $\lceil \log_2(n!) \rceil + (m-1) \cdot \lceil \log_2(n) \rceil$ for the whole representation of the transaction schedule.

3.2 Code Generation Algorithm

The black box function for Grover's search requires qubits as input. Only a limited set of quantum computing operations, i.e., quantum logic gates, can be applied to these qubits. Furthermore, the sequence of quantum computing operations must be fixed in a circuit before the quantum computer starts its processing, i.e., at compile time, such that the quantum computer can be configured accordingly. This is the case for the algorithm for determining a transaction schedule (see Figure 8) whenever the number n of transactions and the number m of machines are fixed at compile time and don't depend on the input, such that e.g. the contained loops can be


```

Algo generateBlackBox
Input:  $n: \mathbb{N}$  // number of transactions
          $l: \mathbb{N}^n$  // lengths of transactions
          $m: \mathbb{N}$  // number of machines
          $c: \mathbb{N}$  // number of conflicts
          $o: \{0, \dots, n-1\}^2 \times c$  // conflicts
          $R: \mathbb{N}$  // maximum runtime on each machine
Output: code

“Algo blackBox”
“Input:  $t: \{\lceil \log_2(n!) \rceil + (m-1) \times \lceil \log_2(n) \rceil$  qubits”
“Output: {True, False}”
“(μ1, ..., μ{m-1}, π) = determineSchedule(t)”
“μm =  $\lceil n-1 \rceil$ ”
// Is this schedule valid?
“result =  $\forall i \in \{2, \dots, \lceil m-1 \rceil\}: \mu_{i-1} < \mu_i$ ”
for(i in 0..n-1)
  “t$i = switch(π[$i])” // determine length of transaction π[i]
  for(j in 0..n-1)
    “case $j: l[$j]”
determineTimes(-1, 1, m, μ1, ..., μm, {s|(s, d) ∈ o ∨ (d, s) ∈ o}, R)
for((o1, o2) ∈ o) // Are conflicting transactions overlapping?
  “if(not(t$o1e ≤ t$o2s or t$o2e ≤ t$o1s)) result = False”
“return result”

Algo determineTimes
Input:  $q: \{0, \dots, n-1\}$  // current transaction in π
          $m_c: \{0, \dots, m-1\}$  // currently considered machine
          $m: \mathbb{N}$  // number of machines
          $\mu_1, \dots, \mu_m: \{0, \dots, n-1\}$  // markers for machines
          $os: \text{set}(\{0, \dots, n-1\})$  // set of transactions in conflicts
          $R: \mathbb{N}$  // threshold of max runtime on each machine
Output: code

if(mc = m or tc = n)
  return
for(i in 0..n-1)
  “if(μ$m_c = $i)”
  “ times = [0, tq+1, tq+1 + tq+2, ...,  $\sum_{r=q+1}^i t_r$ ]”
  // check runtime of this machine
  “ if(times[$i+1] > R) result = False”
  // determine start and end time of conflicting transactions
  for(j in q+1..i)
    for(o ∈ os)
      “ if(π[$j] == $o)”
      “ t$oa = times[$j-1]”
      “ t$oe = times[$j]”
  “ + determineTimes(i, mc+1, m, μ1, ..., μm, os, R)

```

Figure 9: Algorithm *generateBlackBox* and its called algorithm *determineTimes* for generating the code of the black box function for Grover’s search. Generated code is represented by string templates, which might contain $\$v$ and $\{\dots\}$ expressions to be replaced with their computed value. For large sets of conflicting transactions, the assignment of start and end times of these transactions can be further improved by using some kind of decision tree (with $O(\log_2(\min(n, c)))$) over the conflicting transactions (for $n \gg c$) or the transaction numbers (for $c \gg n$) for checking if the considered transaction is a conflicting transaction instead of a sequential check.

```

Algo blackBox
Input:  $t: 7$  qubits
Output: {True, False}
(μ1, π) = determineSchedule(t)
μ2 = 2
result = μ1 < μ2
t0 = switch(π[0])
case 0: 2
case 1: 4
case 2: 1
t1 = switch(π[1])
case 0: 2
case 1: 4
case 2: 1
t2 = switch(π[2])
case 0: 2
case 1: 4
case 2: 1
if(μ1 = 0)
  times = [0, t0]
  if(times[1] > R)
    result = False
  if(π[0] == 0)
    t0a = times[0]
    t0e = times[1]
  if(π[0] == 2)
    t2a = times[0]
    t2e = times[1]
  times = [0, t1, t2]
  if(times[2] > R)
    result = False
  if(π[1] == 0)
    t0a = times[0]
    t0e = times[1]
  if(π[1] == 2)
    t2a = times[0]
    t2e = times[1]
  if(π[2] == 0)
    t0a = times[1]
    t0e = times[2]
  if(π[2] == 2)
    t2a = times[1]
    t2e = times[2]
if(μ1 = 1)
  times = [0, t0, t0 + t1]
  if(times[2] > R)
    result = False
  if(π[0] == 0)
    t0a = times[0]
    t0e = times[1]
  if(π[0] == 2)
    t2a = times[0]
    t2e = times[1]
  if(π[1] == 0)
    t0a = times[1]
    t0e = times[2]
  if(π[1] == 2)
    t2a = times[1]
    t2e = times[2]
  times = [0, t2]
  if(times[1] > R)
    result = False
  if(π[2] == 0)
    t0a = times[0]
    t0e = times[1]
  if(π[2] == 2)
    t2a = times[0]
    t2e = times[1]
  if(not(t0e ≤ t2s or t2e ≤ t0s))
    result = False
return result

```

Figure 10: Example of generated black box for $m = 2$, $n = 3$, $T = [2, 4, 1]$ and $O = \{(0, 2)\}$ in pseudo code. Some compiler optimizations have already been applied like evaluating constant expressions at compile time and dead code elimination.

rolled out already at compile time. Hence the algorithm in Figure 8 can be used within the black box function of Grover's search after changing the input and output types to qubits. Furthermore, due to the limited set of quantum computing operations qubits can e.g. neither be used as indices in arrays nor for indirect variable accesses (like in php `$$name`), such that often a huge set of variables holding values in superposition must be hard coded. Hence there is a need for generating code of the black box function for Grover's search for the specific problem instance (here the configuration of transactions including their lengths and conflicts between each other).

We present the overall code generation algorithm in Figure 9 and an example of a generated black box for Grover's search in Figure 10. Please note that the code sometimes is not as elegant as software developers of modern programming languages are used to, but are due to the limited set of quantum computing operations to be arranged in a circuit. The code generation algorithm is open source and publicly available²: We generate code for the Silq [4] quantum computer programming language and simulator, with which we have extensively tested our generated code.

3.3 Estimating the number of solutions k

We already discussed in subsection 2.2 that Grover's search has a speedup with a factor of $\sqrt{k}$ for the number k of solutions. We have run experiments to determine k with 12 transactions and a length of 10 with standard deviation of 3 varying the number of machines and the number of conflicts (see Figure 11) to show the effects in terms of speedups. N is 8,589,934,592 for $m = 2$ and 2,199,023,255,552 for $m = 4$, but already many optimal solutions exist: 48,384,000 (resulting in a speedup of 6,955) for $m = 2$ and 559,872 (resulting in a speedup of 748) for $m = 4$ ($c = 0$). Determining only suboptimal solutions achieves huge speedups: solutions being close to 25% to the optimal solution can be determined with speedups of 38,374 for $m = 2$ and 45,247 for $m = 4$. Hence we highly recommend to run experiments for typical transaction configurations of the used application and based on these experiments estimate the number of solutions for speeding up Grover's search.

3.4 Complexity Analysis

In this section we compare the complexities according to preprocessing, runtime, space and code size of solving the optimizing transaction schedules problem on a traditional computer, universal quantum computer and quantum annealer. Table 1 summarizes the results of the complexities comparison.

3.4.1 Complexity of Algorithm on Traditional Computer. The job shop scheduling problem (JSSP) as simpler variant of the optimizing transaction schedules problem (without considering blocking transactions) is already among the hardest combinatorial optimization problems [10]. Hence the considered algorithm on the traditional computer enumerates all possible transaction schedules: For each transaction schedule it determines the runtimes in $O(n)$ on each machine to determine the minimum execution time and checks that conflicting transactions do not have overlapping execution times in $O(c)$. There are $n!$ possibilities to reorder the transactions

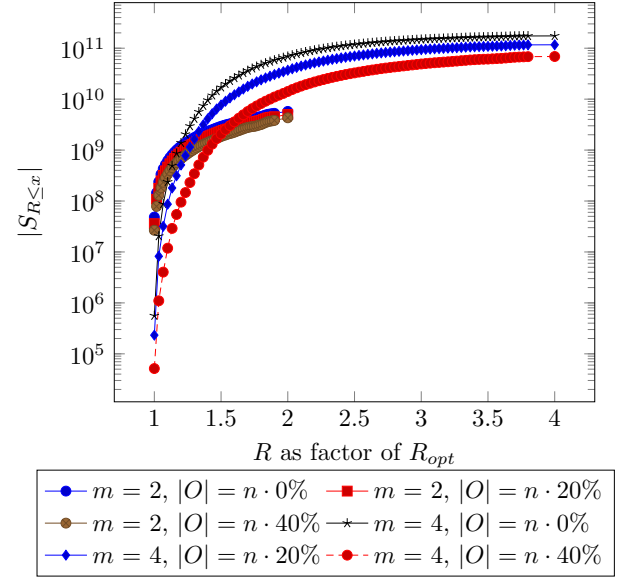


Figure 11: Cumulative number of solutions $|S_{R \leq x}|$ in relation to runtime ($n = 12$, $\sigma_{l_i} = 3$).

(i.e., number of permutations of the n transactions) and the distribution of a permutation of n transactions to m machines is the same as the number of combinations of m objects taken n at a time with repetition, such that $O(n! \cdot \binom{m+n-1}{n})$ possibilities have to be enumerated.

3.4.2 Complexity of Quantum Computing Approach. Grover's search on universal quantum computers takes $O(\sqrt{N})$ iterations, where N is the size of the function's domain and in each iteration the test function needs time $O(n \cdot \log_2(n) + c)$. Please note that determining the length of the i -th transaction in the permutation needs $O(n \cdot \log_2(n))$ time using a decision tree over the transaction numbers. Determining start and end times of conflicting transactions needs $O(n \cdot \log_2(\min(n, c)) + c)$ time by using a decision tree over the conflicting transactions (for $n \gg c$) or the transaction numbers (for $c \gg n$). For our algorithm, $N = O(2^{\log_2(n!) + (m-1) \cdot \log_2(n)}) = O(n! \cdot n^m)$ resulting in an overall runtime complexity of $O(\sqrt{n! \cdot n^m} \cdot (n \cdot \log_2(n) + c))$. The overall runtime complexity can be further improved by estimating the number k of solutions (see subsection 3.3) to $O(\sqrt{\frac{n! \cdot n^m}{k}} \cdot (n \cdot \log_2(n) + c))$, where suboptimal solutions with a higher threshold decreases the runtime further. The space requirement directly depends on the number of used qubits, which is in $O(m \cdot \log_2(n) + \log_2(n!)) = O((n+m) \cdot \log_2(n))$ using Stirling's approximation $n! \sim \sqrt{2 \cdot \pi \cdot n} \cdot (\frac{n}{e})^n$, where $\sim$ means that the two quantities are asymptotic, i.e., their ratio tends to 1 as n tends to infinity.

3.4.3 Complexity of Quantum Annealing Approach. The complexities of the quantum annealing approach have already been determined in [5, 6], but because of simplicity of presentation, the upper bound n^2 for the number c of conflicts have been used in the complexities presented in [5, 6]. Because we directly use the number c of conflicts for Grover's search algorithm in Table 1, we

²<https://github.com/luposdate/OptimizingTransactionSchedulesWithSilq>

	Preprocessing	Execution Time	Space	Code Size
Traditional Computer	$O(1)$	$O(n! \cdot \binom{m+n-1}{n} \cdot (n+c))$ $= O(\frac{(m+n-1)!}{(m-1)!} \cdot (n+c))$	$O(n+c+m)$	$O(1)$
Quantum Computer	$O(n^2 \cdot c)$	$O(\sqrt{\frac{n! \cdot n^m}{k}} \cdot (n \cdot \log_2(n) + c))$	$O((n+m) \cdot \log_2(n))$	$O(n^2 \cdot c)$
Quantum Annealer [5, 6]	$O(m \cdot R^2 \cdot (c \cdot m + n^2))$	$O(1)$	$O(m \cdot R^2 \cdot (c \cdot m + n^2))$	$O(m \cdot R^2 \cdot (c \cdot m + n^2))$

m : number of machines n : number of transactions c : number of conflicts R : maximum execution time on each machine
 k : estimated number of solutions

Table 1: Overview preprocessing, runtime, space and code size complexities of the transaction schedule problem for traditional and different types of quantum computers. The space complexity represents the growth of the number of required qubits for quantum computing, number of variables for quantum annealing and number of bytes for traditional computers.

also determine preciser complexities for the quantum annealing approach resulting in $O(m \cdot R^2 \cdot (c \cdot m + n^2))$ for preprocessing time (i.e., time to generate the formula to optimize), space (i.e., number of binary variables) and code size (i.e., size of the formula to optimize). Once initialized, quantum annealers have a constant execution time $O(1)$.

4 SUMMARY AND CONCLUSIONS

In this paper we propose to use Grover’s search for optimizing transaction schedules on universal quantum computers by generating its black box function based on the given problem instance. We describe the encoding scheme for representing a candidate solution and the code generation. We compare the complexities according to preprocessing, runtime, space and code length with approaches for traditional computers and quantum annealers.

In future work, we will investigate improved encoding schemes for optimizing transaction schedules using less bits, other combinatorial optimization problems and also other applications of Grover’s search like quantum cryptography [17].

REFERENCES

- [1] Adriano Barenco, David Deutsch, Artur Ekert, and Richard Jozsa. 1995. Conditional quantum dynamics and logic gates. *Physical Review Letters* 74, 20 (1995).
- [2] William P. Baritompa, David W. Bulger, and Graham R. Wood. 2005. Grover’s quantum algorithm applied to global optimization. *SIAM Journal on Optimization* 15, 4 (2005).
- [3] Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. 1997. Strengths and Weaknesses of Quantum Computing. *SIAM J. Comput.* 26, 5 (1997), 1510–1523. <https://doi.org/10.1137/S0097539796300933>
- [4] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 286–300.
- [5] Tim Bittner and Sven Groppe. 2020. Avoiding Blocking by Scheduling Transactions Using Quantum Annealing. In *IDEAS*.
- [6] Tim Bittner and Sven Groppe. 2020. Hardware Accelerating the Optimization of Transaction Schedules via Quantum Annealing by Avoiding Blocking. *Open Journal of Cloud Computing (OJCC)* 7, 1 (2020).
- [7] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. 1998. Tight Bounds on Quantum Searching. *Fortschritte der Physik* 46, 4-5 (1998).
- [8] Antoine Comeau. 2014. Mapping between permutations and natural numbers. Blog available at <http://antoinecomeau.blogspot.com/2014/07/mapping-between-permutations-and.html>.
- [9] Thomas M Connolly and Carolyn E Begg. 2005. *Database systems: a practical approach to design, implementation, and management*. Pearson Education.
- [10] M. R. Garey, D. S. Johnson, and Ravi Sethi. 1976. The Complexity of Flowshop and Jobshop Scheduling. *Mathematics of Operations Research* 1, 2 (1976), 117–129.
- [11] Sven Groppe. 2020. Emergent models, frameworks, and hardware technologies for Big data analytics. *The Journal of Supercomputing* 76, 3 (2020), 1800–1827. <https://doi.org/10.1007/s11227-018-2277-x>
- [12] Sven Groppe. 2021. Semantic Hybrid Multi-Model Multi-Platform (SHM3P) Databases. In *International Semantic Intelligence Conference (ISIC 2021)*, New Delhi (hybrid), India. <http://ceur-ws.org/Vol-2786/Paper2.pdf>
- [13] Sven Groppe and Jinghua Groppe. 2020. Hybrid Multi-Model Multi-Platform (HM3P) Databases. In *Proceedings of the 9th International Conference on Data Science, Technology and Applications (DATA)*.
- [14] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *STOC*.
- [15] Lov K Grover. 1997. Quantum computers can search arbitrarily large databases by a single query. *Physical review letters* 79, 23 (1997).
- [16] Derrick H Lehmer. 1960. Teaching combinatorial tricks to a computer. In *Proc. Sympos. Appl. Math. Combinatorial Analysis*, Vol. 10. 179–193.
- [17] Diana Maimut and Emil Simion. 2018. Post-quantum Cryptography and a (Qu)Bit More. In *SecITC*.
- [18] Kyoichi Okamoto and Osamu Watanabe. 2001. Deterministic Application of Grover’s Quantum Search Algorithm. In *COCOON*.
- [19] Peter W Shor. 1994. Algorithms for quantum computation: Discrete logarithms and factoring. In *Symposium on foundations of computer science*.
- [20] A. Thomson, T. Diamond, S.-C. Weng, K. Ren, P. Shao, and D. J. Abadi. 2012. Calvin: Fast Distributed Transactions for Partitioned Database Systems. In *SIGMOD*.